Quy trình RAG (Retrieval-Augmented Generation)

Oct 11, 2026 · @Hung Ta Quang

Giới thiệu

RAG (Retrieval-Augmented Generation – Sinh văn bản tăng cường bằng truy xuất) là kỹ thuật cho LLM trả lời dựa trên tài liệu riêng của tổ chức, thay vì chỉ dựa vào kiến thức đã học khi huấn luyện. Hệ thống tìm các đoạn tài liệu liên quan đến câu hỏi, ghép chúng vào prompt làm ngữ cảnh, rồi mới để LLM sinh câu trả lời.

RAG giải quyết ba hạn chế của LLM thuần:

- **Kiến thức cũ:** LLM không biết thông tin sau thời điểm huấn luyện; RAG cập nhật bằng cách thêm tài liệu mới, không cần huấn luyện lại.
- **Không biết dữ liệu nội bộ:** quy chế, văn bản, cơ sở dữ liệu riêng của đơn vị chưa từng có trong dữ liệu huấn luyện.
- **Bịa thông tin (hallucination):** khi có ngữ cảnh đúng, câu trả lời bám sát nguồn và có thể trích dẫn lại.

Quy trình 5 bước

Mỗi câu hỏi của người dùng đi qua 5 bước, từ lúc nhập câu hỏi đến lúc nhận câu trả lời:

1. **Người dùng nhập câu hỏi (query)** qua ứng dụng (web, chatbot, Moodle…).
2. **Retriever truy xuất thông tin liên quan:** câu hỏi được chuyển thành vector, rồi so khớp với các vector trong Vector Database để lấy ra các đoạn tài liệu gần nghĩa nhất.
3. **Bổ sung ngữ cảnh vào prompt (augment):** các đoạn tài liệu tìm được được ghép cùng câu hỏi thành một prompt hoàn chỉnh: Prompt = Context + Query.
4. **LLM sinh câu trả lời** dựa trên prompt, ưu tiên thông tin trong phần ngữ cảnh.
5. **Trả kết quả cho người dùng**, thường ở dạng streaming (hiển thị dần từng phần).

Các thành phần

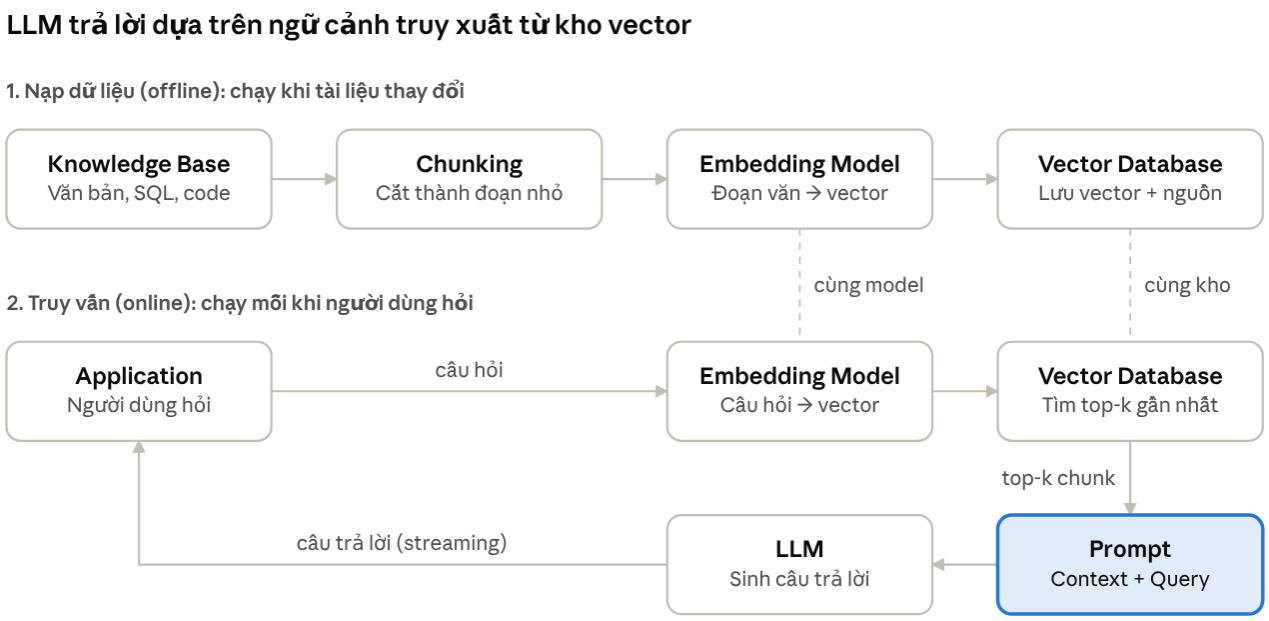
Kiến trúc RAG gồm 7 thành phần; bảng dưới nêu vai trò và công cụ thường dùng cho từng thành phần.

Thành phần	Vai trò	Đầu vào → Đầu ra	Công cụ ví dụ
Knowledge Base	Nguồn tri thức gốc của hệ thống	Tài liệu (PDF, DOCX), dữ liệu SQL, mã nguồn	Thư mục văn bản, CSDL nội bộ
Chunking	Cắt tài liệu thành các đoạn nhỏ vừa đủ ngữ nghĩa	Tài liệu → các chunk	LangChain text splitter, tự viết bằng Python
Embedding Model	Chuyển văn bản thành vector số thể hiện ý nghĩa	Chunk hoặc câu hỏi → vector	nomic-embed-text, bge-m3 (qua Ollama)
Vector Database	Lưu vector và tìm các vector gần nhất với câu hỏi	Vector truy vấn → top-k chunk liên quan	ChromaDB, FAISS, Qdrant, pgvector
Application	Giao diện nhận câu hỏi, điều phối luồng xử lý	Câu hỏi của người dùng → prompt	Web app, chatbot, plugin Moodle
Prompt	Ghép ngữ cảnh truy xuất với câu hỏi	Context + Query → prompt hoàn chỉnh	Prompt template
LLM	Sinh câu trả lời từ prompt	Prompt → câu trả lời (streaming)	Llama, Qwen, Gemma (qua Ollama); Claude, GPT (API)

Trong Vector Database ở hình, chấm xanh lá là vector câu hỏi, các chấm trong vòng tròn "Context" là vector tương tự được chọn làm ngữ cảnh, còn lại là các vector đã lưu nhưng không liên quan.

Hai luồng xử lý

Hệ thống RAG chạy hai luồng tách biệt: luồng nạp dữ liệu chuẩn bị kho vector trước, luồng truy vấn dùng kho đó cho từng câu hỏi.



kiến trúc RAG · 2 luồng xử lý

Luồng 1 chỉ chạy lại khi tài liệu thay đổi; luồng 2 chạy cho mỗi câu hỏi và chỉ đọc từ kho vector. Hai luồng phải dùng chung một embedding model thì vector câu hỏi và vector tài liệu mới so sánh được.

Lưu ý khi triển khai

Chất lượng câu trả lời của RAG phụ thuộc chủ yếu vào khâu truy xuất: lấy sai đoạn thì LLM tốt đến đâu cũng trả lời sai.

- **Kích thước chunk:** thường 300–1.000 ký tự, chồng lấn (overlap) 10–20% để không cắt đứt ý. Văn bản quy chế nên cắt theo điều, khoản thay vì cắt cứng theo số ký tự.
- **Giữ metadata:** lưu kèm tên văn bản, số điều, ngày ban hành cho mỗi chunk để trích dẫn nguồn và lọc kết quả.
- **Dùng cùng một embedding model** cho cả tài liệu và câu hỏi; đổi model thì phải embedding lại toàn bộ kho.
- **Hỗ trợ tiếng Việt:** chọn embedding model đa ngôn ngữ (ví dụ bge-m3) và kiểm tra thực tế trên câu hỏi tiếng Việt.
- **Số đoạn truy xuất (top-k):** thường 3–5; quá ít dễ thiếu thông tin, quá nhiều gây nhiễu và tốn token.
- **Ràng buộc trong prompt:** yêu cầu LLM chỉ trả lời theo ngữ cảnh, nói "không tìm thấy" khi ngữ cảnh không có thông tin, và ghi nguồn.
- **Cập nhật dữ liệu:** khi văn bản thay đổi, xóa chunk cũ và nạp lại để tránh trả lời theo quy định đã hết hiệu lực.

- **Đánh giá:** chuẩn bị bộ câu hỏi mẫu có đáp án để đo độ chính xác sau mỗi lần thay đổi cấu hình.

Thuật ngữ

Thuật ngữ	Giải thích
Query	Câu hỏi người dùng nhập vào
Chunk	Một đoạn nhỏ được cắt ra từ tài liệu gốc
Embedding	Vector số biểu diễn ý nghĩa của một đoạn văn bản
Vector Database	CSDL chuyên lưu và tìm kiếm vector theo độ tương đồng
Similarity search	Tìm các vector gần nhất (thường đo bằng cosine similarity)
Retriever	Bộ phận tìm và trả về các chunk liên quan đến câu hỏi
Context	Các chunk được truy xuất, đưa vào prompt cho LLM
Prompt	Toàn bộ nội dung gửi cho LLM: chỉ dẫn + ngữ cảnh + câu hỏi
LLM	Mô hình ngôn ngữ lớn, sinh câu trả lời
Streaming	Trả kết quả dần từng phần thay vì chờ sinh xong toàn bộ
Hallucination	LLM tạo ra thông tin sai nhưng trông hợp lý